NASCOMSOFT EMBEDDED LTD

SOFTWARE DEVELOPMENT WITH PYTHON

WEEK 2 - DAY 1

Introduction to Web APIs, HTTP & Client-Server Communication

Student Self-Learning Lecture Handbook

This chapter is designed to be studied independently. Work through it in order, complete the quick checks, and use the Task Management API example as the thread connecting the concepts.

How to Use This Handbook

This is not a set of abbreviated lecture notes. It is a guided chapter that introduces the vocabulary and mental models you need before you begin writing FastAPI code on Day 2. Read the explanations, trace the diagrams, and attempt exercises before checking the answer key.

- Do not rush to memorize terms. Ask what role each idea plays in a real request-response conversation.
- Keep returning to the Task Management API examples. The same resource will be used for methods, endpoints, parameters, JSON and status codes.
- When you see an HTTP snippet, read it line by line.
- Complete the quick checks without looking ahead.
- The final self-assessment is deliberately application-focused, not only definition-focused.

PREREQUISITE

You are expected to know basic Python syntax, functions, collections, exceptions, modules and introductory OOP. You are not expected to know backend development or HTTP yet.

Learning Objectives

- Explain what a web application is and distinguish client and server roles.
- Describe client-server architecture and how web applications communicate over a network.
- Define an API and explain why APIs are useful.
- Explain REST at the level needed to design simple resource-oriented APIs.
- Read the main parts of an HTTP request and response.
- Choose appropriate HTTP methods for CRUD operations.
- Identify URLs, endpoints, path parameters, query parameters, request bodies and HTTP headers.
- Read and write basic JSON objects and arrays.
- Interpret common HTTP status codes used in this course.
- Trace a complete API request-response lifecycle and explain where FastAPI fits.

1. What Is a Web Application?

Concept

A web application is software whose components communicate using web technologies, normally across a network. A web application may have a user interface, a backend, and a data store. These parts can run on different machines and still behave as one product from the user's point of view.

What this means in practice

When you use a task-management website, the page in your browser does not need to contain every task permanently. It can ask a backend service for the latest tasks. The backend can retrieve the information, apply rules, and return data to the browser.



Simplified web-application architecture

WHY THIS MATTERS

As a backend developer, you will mostly build the server-side/API part. To build it correctly, you must understand what the client sends, what the server receives, and what the server must return.

COMMON MISTAKE

Thinking that "web application" means only the visible website. The user interface is one part; backend services and data storage are also part of the system.

QUICK CHECK

1. In a task-management website, which part usually displays buttons and forms to the user?
2. Which part usually applies rules such as "task title is required"?

2. Client and Server

Client

A client is software that initiates a request for a service or resource. In our course examples, a browser, mobile app, Swagger UI, Postman, or another program can act as a client.

Server

A server is software that waits for incoming requests, processes them, and returns responses. A FastAPI application running through a server such as Uvicorn will act as the server side of our API.



The request-response conversation

TASK MANAGEMENT EXAMPLE

The mobile app sends: "Give me task 12." The backend receives the request, looks for task 12, and

returns either the task data or an error response such as 404 Not Found.

COMMON MISTAKE

Client and server describe roles, not permanent types of machines. The same computer can run a browser (client) and a local FastAPI server during development.

QUICK CHECK

1. If Swagger UI sends GET /tasks to your FastAPI app, which side is the client?
2. Can the client and server run on the same laptop during development?

3. How Applications Communicate Over a Network

For two applications to communicate, they need an address and a shared set of communication rules. On the web, HTTP provides the request-response rules. The client targets a server address, sends a request, and waits for a response.

1. The client identifies the server location (for example, api.example.com or localhost during development).
2. The client chooses an HTTP method and target path.
3. The client may add headers and a request body.
4. The server receives the request and selects matching backend logic.
5. The backend processes the request and may read or modify data.
6. The server sends an HTTP response containing a status code, headers and possibly a body.
7. The client interprets the response and updates its interface or takes another action.

IMPORTANT

You do not need advanced networking theory for this course. Focus on the application-level conversation: address, request, processing and response.

4. What Is an API?

Definition

API stands for Application Programming Interface. An API is a defined interface that allows one software system to interact with another using agreed operations, inputs and outputs.

Why APIs are needed

Without an agreed interface, a client would need to know too much about the server's internal implementation. APIs create a boundary: the client asks for supported operations, and the server controls how those operations are performed internally.

Application A	→	API contract	→	Application B

REAL-WORLD ANALOGY

Think about ordering in a restaurant. You use a menu to see what operations are available and tell a waiter what you want. You do not walk into the kitchen and control the cooking process. The menu and ordering process are similar to an interface: they expose allowed interactions while hiding internal work.

LIMIT OF THE ANALOGY

An API is not literally a waiter. Real APIs have precise machine-readable paths, methods, data formats, status codes and validation rules.

Why backend developers care

- You must define operations clients are allowed to perform.
- You must define what data clients send and receive.
- You must return predictable responses so frontend developers can integrate reliably.
- You must protect internal details such as direct database access.

QUICK CHECK

1. What does API stand for?
2. Why is an API better than allowing a mobile app to connect directly to the database?

5. REST APIs - The Level You Need for This Course

REST is a widely used architectural style for web APIs. This course will not study advanced REST theory. The practical idea you need is to design endpoints around resources and use HTTP methods to express what should happen to those resources.

What is a resource?

A resource is something meaningful in the application domain that the API exposes. In our running example, Task is a resource. Other projects might expose Book, Expense, Product, Complaint or Appointment resources.

```
GET    /tasks
POST   /tasks
```

```
GET    /tasks/1
PUT    /tasks/1
PATCH  /tasks/1
DELETE /tasks/1
```

GOOD DESIGN HABIT

Use nouns for resources in paths. Prefer GET /tasks instead of GET /getAllTasks. The HTTP method already communicates the action.

QUICK CHECK

1. What is the resource in GET /tasks/5?
2. Which path is more resource-oriented: POST /createTask or POST /tasks?

6. HTTP - The Language of the Web Conversation

HTTP stands for Hypertext Transfer Protocol. It is an application-layer protocol that defines how a client and server format requests and responses. For API development, HTTP gives us concepts such as methods, paths, headers, bodies and status codes.

WHY IT MATTERS

FastAPI is a Python framework, but the clients talking to it use HTTP. If you only learn FastAPI syntax without understanding HTTP, route decorators and status codes will feel like arbitrary framework rules.

COMMON MISTAKE

HTTP is not your application logic. HTTP defines the communication envelope. Your Python code still decides whether a task can be created, how data is validated, and what business rules apply.

7. HTTP Requests and Responses

HTTP request

A request travels from client to server. It identifies an action and target, and may include metadata and data.

```
POST /tasks HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

```
{
  "title": "Prepare report",
  "completed": false
}
```

Read the request from top to bottom:

- POST is the HTTP method.
- /tasks is the target path.
- Host identifies the server host.
- Content-Type tells the server the request body contains JSON.
- The blank line separates headers from the body.
- The JSON object is the data sent to the server.

HTTP response

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 3,
  "title": "Prepare report",
  "completed": false
}
```

- 201 Created is the status code and reason phrase.
- Content-Type describes the response body format.
- The JSON object is the representation returned to the client.

COMMON MISTAKE

A request does not have a response status code. Status codes belong to responses.

QUICK CHECK

1. In POST /tasks, which part tells the server what action the client wants?
2. Which response element tells the client whether the operation succeeded or failed?

8. HTTP Methods

The HTTP method communicates the intended type of operation. The five methods below will cover most of the work in this course.

Method	Typical use	Task API example
GET	Retrieve data without	GET /tasks

	intending to modify the resource.	
POST	Submit data, commonly to create a new resource.	POST /tasks
PUT	Replace the current representation of a resource with a supplied representation.	PUT /tasks/1
PATCH	Apply a partial change to a resource.	PATCH /tasks/1
DELETE	Remove a resource.	DELETE /tasks/1

GET

Use GET when the client wants to retrieve data. Example: GET /tasks retrieves a collection; GET /tasks/7 retrieves task 7.

```
GET /tasks
GET /tasks/7
```

POST

Use POST when the client submits data to create a new resource in our course examples.

```
POST /tasks

{
 "title": "Prepare report"
}
```

PUT vs PATCH

Both are update methods. For this course, use PUT when the client sends the full updated representation expected by the API and PATCH when it changes only selected fields.

```
PUT /tasks/7
{
 "title": "Prepare report",
 "description": "Send today",
 "completed": false
}

PATCH /tasks/7
{
 "completed": true
```

```
}
```

DELETE

DELETE requests removal of a resource, for example DELETE /tasks/7.

COMMON MISTAKE

Do not choose a method just because its English name sounds convenient. Choose based on the API operation. For example, GET should not be used to delete a task.

QUICK CHECK

1. Which method should create a task?
2. Which method should retrieve task 20?
3. Which method fits changing only completed from false to true?

9. CRUD and Its Relationship with HTTP

CRUD is a software-development acronym for four basic data operations: Create, Read, Update and Delete. HTTP methods give us a natural way to express these operations in a web API.

CRUD	Meaning	HTTP	Example
Create	Add data	POST	POST /tasks
Read	Retrieve data	GET	GET /tasks/1
Update	Change data	PUT / PATCH	PATCH /tasks/1
Delete	Remove data	DELETE	DELETE /tasks/1

PRACTICE

Requirement: "A user marks a task completed." CRUD operation: Update. Suitable HTTP method: PATCH if only completed is changing. Endpoint: PATCH /tasks/{task_id}.

10. URLs and API Endpoints

```
https://api.example.com/tasks/1?details=true
```

A URL can contain several useful parts:

- https - scheme/protocol indicator.

- api.example.com - host (server name).
- /tasks/1 - path.
- ?details=true - query string.

What is an endpoint?

In practical API work, an endpoint is an exposed API operation identified by an HTTP method and path. GET /tasks and POST /tasks are different endpoints because the methods differ.

COMMON MISTAKE

Treating the path alone as the complete operation. The method matters. GET /tasks and DELETE /tasks would mean very different things.

11. Path Parameters and Query Parameters

Path parameters

A path parameter is a variable value embedded in the path. It commonly identifies a specific resource.

```
GET /tasks/{task_id}

Examples:
GET /tasks/1
GET /tasks/25
```

For GET /tasks/25, the value of task_id is 25.

Query parameters

Query parameters are key-value values placed after ? in a URL. They commonly filter, sort, search or otherwise modify how a resource collection is retrieved.

```
GET /tasks?completed=true
GET /tasks?completed=false&limit=10
```

In the second example, completed and limit are query parameter names. Query parameters are separated using & when more than one is present.

	Path parameter	Query parameter
Question answered	Which specific resource/value?	How should the request be modified/filter results?
Example	/tasks/42	/tasks?completed=true

Location	Inside the path	After ?

QUICK CHECK

1. What is the path parameter value in GET /tasks/88?
2. Which is more suitable for "show only completed tasks": /tasks/completed/true or /tasks?completed=true? Explain your choice.

12. Request Bodies and JSON

Request body

A request body carries data inside the request. In this course, POST, PUT and PATCH operations will commonly send bodies.

```
POST /tasks
Content-Type: application/json

{
  "title": "Prepare report",
  "description": "Send by 4 PM"
}
```

JSON

JSON (JavaScript Object Notation) is a text data format commonly used by web APIs. It is language-independent even though its syntax resembles Python dictionaries and lists.

```
{
  "id": 1,
  "title": "Prepare report",
  "completed": false,
  "tags": ["work", "urgent"]
}
```

Important JSON structures and values:

- Object: { } containing key-value pairs.
- Array: [ ] containing ordered values.
- String: enclosed in double quotation marks.
- Number: integer or decimal number.
- Boolean: true or false (lowercase in JSON).
- Null: null.

COMMON MISTAKE

Python True, False and None are not valid JSON literals. JSON uses true, false and null.

OBJECT VS ARRAY

A task is naturally represented as one JSON object. A collection of tasks is commonly represented as a JSON array containing several task objects.

QUICK CHECK

1. Is {"title": "Study HTTP"} an object or array?
2. What is wrong with {"completed": True} if it is intended to be JSON?

13. HTTP Headers and Content-Type

Headers carry metadata about a request or response. Metadata describes the message rather than being the main application data itself.

```
Content-Type: application/json
Accept: application/json
```

Content-Type

Content-Type tells the receiver how to interpret the message body. For JSON bodies, you will commonly see application/json.

WHY THIS MATTERS

If the server expects JSON, it needs to know that the body should be interpreted as JSON. Frameworks such as FastAPI use request metadata and endpoint definitions to process incoming data.

NOT YET

Later in the course, HTTP headers will also carry authentication credentials such as bearer tokens. Authentication is intentionally outside today's lesson.

14. HTTP Status Codes

A status code is a three-digit number in the HTTP response that communicates the overall result of processing a request. Clients should not have to inspect a human sentence to determine whether a request succeeded.

Status categories used today

- 2xx - successful processing.
- 4xx - the request cannot be fulfilled because of a client/request-side issue, such as missing data, missing authentication or a resource that does not exist.
- 5xx - the server failed while handling a request that it should otherwise be capable of processing.

Status	Meaning	Task API example
200 OK	The request succeeded.	GET /tasks returns tasks.
201 Created	A new resource was successfully created.	POST /tasks creates task 8.
204 No Content	The request succeeded and no response body is returned.	DELETE /tasks/8 succeeds.
400 Bad Request	The request is invalid at a general level.	The server cannot accept the submitted request.
401 Unauthorized	Authentication is required or credentials are invalid.	Used later for protected endpoints.
404 Not Found	The requested resource does not exist.	GET /tasks/999 when no task 999 exists.
422 Unprocessable Entity	The request can be read but fails validation/semantic requirements.	FastAPI commonly uses this for request-validation errors.
500 Internal Server Error	An unexpected server-side failure occurred.	Unhandled exception while processing a request.

401 NAME CAN BE CONFUSING

Although the reason phrase is "Unauthorized", 401 is normally used when authentication is missing or invalid. Authorization/permission issues are commonly represented differently, but detailed authentication is a later lesson.

422 IN THIS COURSE

We will use the curriculum wording "422 Unprocessable Entity" and associate it with validation failures, which is a common FastAPI experience for beginners.

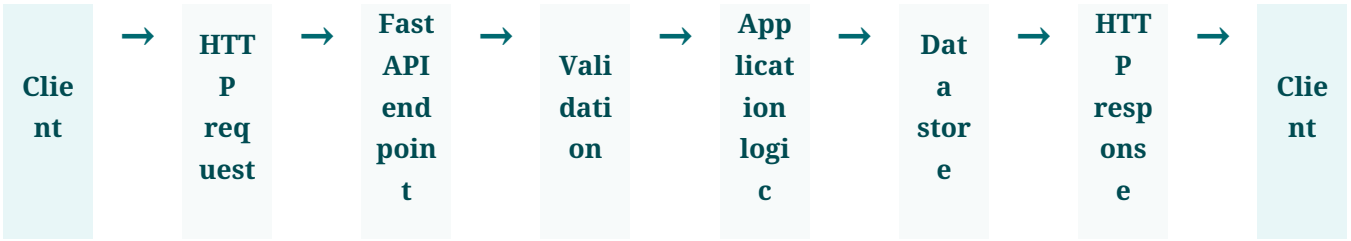
QUICK CHECK

1. A new task is created successfully. Which status code best communicates this?
2. A client asks for task 900 but it does not exist. Which code?
3. The server crashes unexpectedly while processing a valid request. Which category: 2xx, 4xx or

5xx?

15. The Complete API Request-Response Lifecycle

Now connect all the separate concepts. Imagine a client creating a new task.



8. The client prepares POST /tasks.
9. It adds Content-Type: application/json and a JSON request body.
10. The request reaches the server.
11. FastAPI matches the POST method and /tasks path to the correct endpoint.
12. The input is validated against the expected data shape.
13. If valid, application logic creates the task and later will save it to a database.
14. The server builds an HTTP response, for example 201 Created plus the created task as JSON.
15. The client reads the status code and response body, then updates its interface.

```
POST /tasks
Content-Type: application/json

{ "title": "Prepare report" }

↓

201 Created
Content-Type: application/json

{ "id": 3, "title": "Prepare report", "completed": false }
```

IMPORTANT FOR DAY 2

Tomorrow you will implement the endpoint and see FastAPI perform routing and validation. Databases remain a later concern, so initial data may be stored only in memory.

16. Where FastAPI Fits

FastAPI is a Python web framework for building APIs. It sits on the server side between HTTP requests and your Python application logic. It helps route requests to Python functions, extract parameters, validate data, and construct responses.



A preview, not today's main coding task

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello World"}
```

Read this conceptually:

- FastAPI() creates the API application object.
- @app.get("/") declares an endpoint that handles GET requests at /.
- home() is the Python function that runs when that endpoint is called.
- Returning a Python dictionary allows FastAPI to produce a JSON response.

DAY 2 BRIDGE

On Day 2, you will install/use FastAPI and Uvicorn, start a development server, open the automatic API documentation, and build the first real routes.

17. Guided Practice Exercises

Exercise A - Identify client and server

16. A browser requests https://example.com/tasks from a backend. Identify the client and server.
17. Swagger UI sends POST /tasks to a locally running FastAPI app. Identify the client and server.
18. A mobile application sends GET /weather to a weather service. Identify the two roles.

Exercise B - Choose the HTTP method

19. Retrieve all tasks.
20. Create a task.
21. Retrieve task 12.
22. Replace task 12 with a complete updated representation.
23. Change only task 12 completed status.

24. Delete task 12.

Exercise C - Identify endpoint and parameters

```
GET /tasks/15?details=true
```

25. What is the HTTP method?
26. What is the path?
27. What is the path parameter value?
28. What is the query parameter name and value?

Exercise D - Analyze an HTTP request

```
PATCH /tasks/15 HTTP/1.1
Host: api.example.com
Content-Type: application/json

{ "completed": true }
```

29. What operation is the client attempting?
30. What resource is targeted?
31. Which header describes the request body format?
32. What JSON value is being changed?

Exercise E - Choose a status code

33. GET /tasks/4 succeeds and returns the task.
34. POST /tasks successfully creates task 8.
35. DELETE /tasks/8 succeeds and the API returns no body.
36. GET /tasks/999 finds no such task.
37. POST /tasks is missing a required title and fails validation.
38. An unexpected server exception occurs.

18. Larger Practical Exercise - Design a Simple API

Choose one of these systems: Expense Tracker, Library Management, Inventory Management, Appointment Management, Complaint Management, or another approved project from Week 1.

Choose one main resource. Then design an API without writing FastAPI code yet.

39. Name the resource and give a sample JSON object for it.
40. Design endpoints for Create, Read all, Read one, Update and Delete.
41. Choose PUT or PATCH for at least one update and explain why.
42. Add one query parameter that would be useful for filtering.
43. For each endpoint, choose a success status code.
44. Write one 404 scenario and one validation-error scenario.
45. Draw the request-response lifecycle for one of your endpoints.

EXAMPLE START

Resource: Expense. Possible collection path: /expenses. A useful query could be GET /expenses?category=transport.

19. Chapter Recap

- A client initiates a request; a server processes it and responds.
- An API defines supported interactions between software systems.
- REST-style APIs commonly expose resources through paths and use HTTP methods to represent operations.
- HTTP requests use methods, paths, headers and sometimes bodies.
- HTTP responses use status codes, headers and sometimes bodies.
- GET reads, POST creates, PUT/PATCH update and DELETE removes in our course examples.
- CRUD is a general software concept; HTTP methods map naturally to CRUD for APIs.
- Path parameters identify values inside the path; query parameters modify/filter requests.
- JSON is a common API data format.
- Content-Type: application/json identifies a JSON body.
- 2xx indicates success, 4xx a request/client-side problem, and 5xx a server problem.
- FastAPI will implement these concepts using Python on Day 2.

20. Self-Assessment

Attempt all questions before reading the answer key. Explain your reasoning wherever possible.

Conceptual

1. Explain the difference between a client and a server using a task-management app.

2. What problem does an API solve?

3. In simple terms, what is REST trying to organize around?

4. What is HTTP responsible for, and what remains the responsibility of application logic?

5. Why is an HTTP status code useful even if the response body contains a message?

Methods and CRUD

6. Choose the method for retrieving all tasks.

7. Choose the method for creating a task.

8. Choose the method for completely replacing task 4.

9. Choose the method for changing only task 4 completed status.

10. Choose the method for removing task 4.

11. Map Create, Read, Update and Delete to the common HTTP methods used in this course.

Endpoints and parameters

12. In GET /tasks/27, identify the method, path and path-parameter value.

13. In GET /tasks?completed=true&limit=5, identify both query parameters.

14. Explain one difference between a path parameter and a query parameter.

15. Are GET /tasks and POST /tasks the same endpoint operation? Explain.

JSON and request anatomy

16. Is {"title": "Study HTTP"} a JSON object or array?

17. Correct this JSON: {"completed": True, "note": None}.

18. In a POST request carrying JSON, what header should commonly appear?

19. In the request line PATCH /tasks/5 HTTP/1.1, what does PATCH communicate? What does /tasks/5 communicate?

Status codes

20. A GET request succeeds. Choose an appropriate status code.

21. A POST creates a task. Choose the best status code.

22. A DELETE succeeds and returns no body. Choose a status code.

23. Task 100 does not exist. Choose a status code.

24. A submitted request fails validation because title is missing. Which course status code fits?

25. An unexpected backend exception prevents the server from completing the request. Which code fits?

Design

26. Design five CRUD endpoints for a Book resource.

27. Add a query parameter for showing only available books.

28. Write a JSON object for one book.

29. Choose the likely success status code for creating the book.

30. Trace the lifecycle of POST /books from client to response in at least six steps.

21. Answer Key

1. The client (browser/mobile app) starts the request; the server/backend receives it, processes it and responds. 2. An API gives software systems a defined interface/contract instead of requiring direct access to internal implementation. 3. REST-style design organizes operations around resources. 4. HTTP structures the communication; application logic decides business behavior. 5. Status codes give a standard machine-readable outcome.

6. GET. 7. POST. 8. PUT. 9. PATCH. 10. DELETE. 11. Create→POST, Read→GET, Update→PUT/PATCH, Delete→DELETE.

12. Method GET; path /tasks/27; path-parameter value 27. 13. completed=true and limit=5. 14. Path parameters are part of the path and commonly identify a specific resource; query parameters appear after ? and commonly filter/modify the request. 15. No. Same path, different HTTP method and purpose.

16. Object. 17. {"completed": true, "note": null}. 18. Content-Type: application/json. 19. PATCH expresses a partial-update operation; /tasks/5 identifies the target resource path.

20. 200 OK. 21. 201 Created. 22. 204 No Content. 23. 404 Not Found. 24. 422 Unprocessable Entity. 25. 500 Internal Server Error.

26. Example: POST /books, GET /books, GET /books/{book_id}, PUT or PATCH /books/{book_id}, DELETE /books/{book_id}. 27. GET /books?available=true. 28. Example: {"id": 1, "title": "Python Basics", "available": true}. 29. 201 Created. 30. A complete answer should include client builds request → HTTP request sent → endpoint matched → input validated → application logic runs → data stored (conceptually) → HTTP response built → client receives and interprets it.

22. Preparing for Day 2

Day 2 moves from understanding to implementation. You will begin building the Task Management API with FastAPI and run it using Uvicorn. Before class, make sure you can explain the following without looking at the answer key:

- What a client and server are.
- What GET /tasks means.
- What POST /tasks means.
- The difference between path and query parameters.
- What JSON is.
- Why Content-Type matters.
- When 200, 201, 204, 404, 422 and 500 are used.
- How an HTTP request becomes an HTTP response.

NEXT CLASS

You will see these concepts become code: app = FastAPI(), route decorators such as @app.get(...),

path/query parameters, JSON responses, automatic documentation and Uvicorn serving the application.

Reference notes: Terminology and examples are aligned with standard HTTP usage and the official FastAPI documentation. This handbook intentionally limits itself to the concepts required for the course sequence.